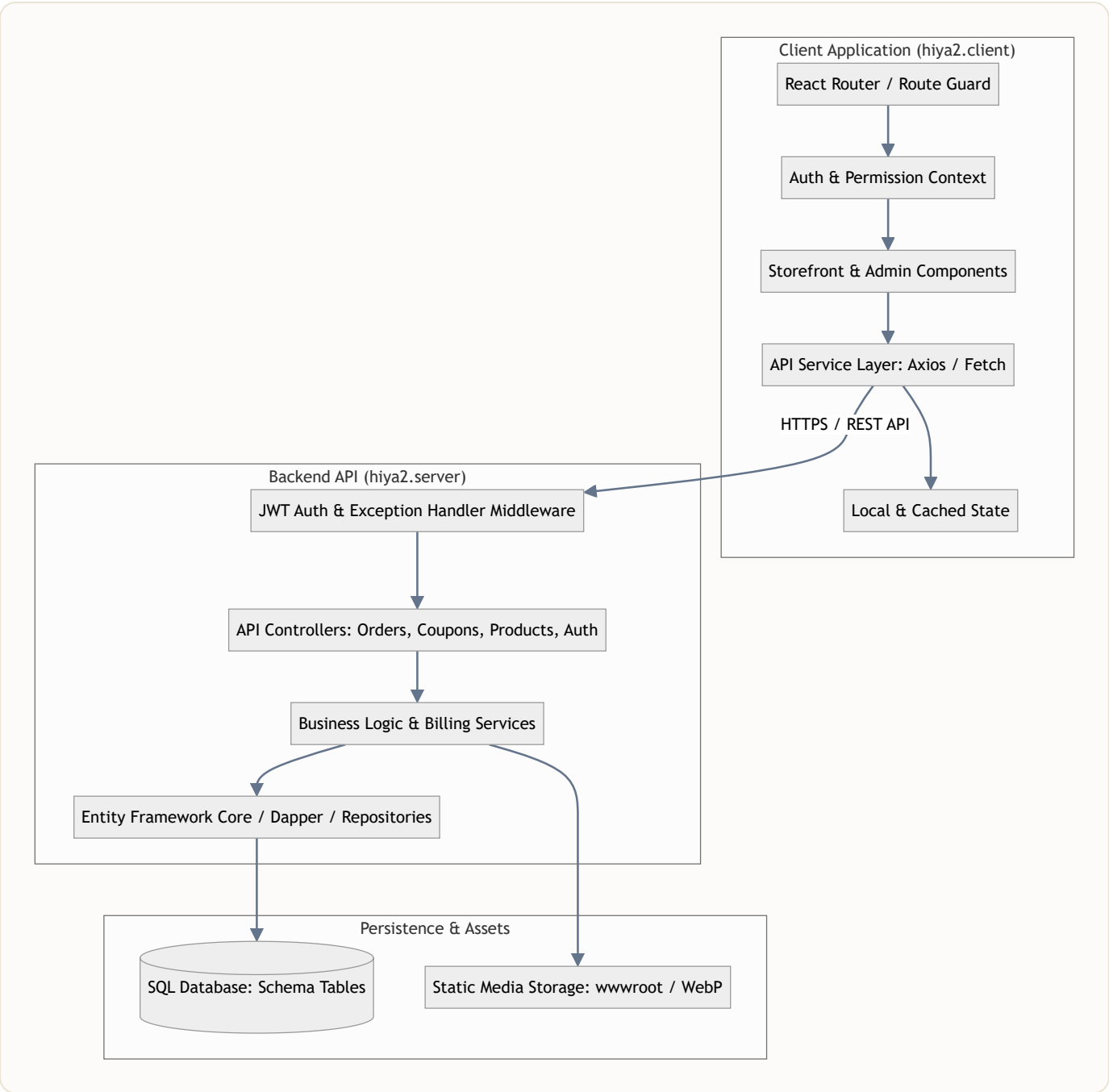


HiyaGhar — End-to-End Developer Architecture & Engineering Workflow

Technical Specification & System Design Manual for Engineers & Tech Leadership

1. System Architecture Overview

HiyaGhar is structured as a decoupled, multi-tier architecture with a high-performance **React 18 + Vite (SPA)** client communicating over JSON REST APIs to an **ASP.NET Core Web API** backend connected to an enterprise SQL database.



2. Directory & Module Organization

HiyaGhar-main/

hiya2.client/

e2e-tests/

public/

src/

assets/

components/

admin/

common/

home/

layout/

mukhwas/

context/

pages/

Admin/

Cart/

Checkout/

ProductDetail/

Profile/

services/

utils/

hiya2.server/

Controllers/

Models/

Services/

Data/

wwwroot/

React + TypeScript Frontend

Playwright Automated Test Matrix (38 cases)

Static public assets (Favicon, Logo, Images)

Vector icons, CSS utilities, design tokens

AdminLayout, DataTable, ModalCard, Nav

Header, Footer, ToastAlert, FloatingWidgets

Banner, ComboCreator, BestSellers, Reviews

Global wrappers, SEO head, Navigation

ProductCards, CategoryFilters, Badges

AuthContext, PermissionContext, CartContext

Dashboard, Orders, Products, Coupons, Roles

CartPage, BillBreakdown, CouponInput

Multi-step checkout, AddressForm, Payment

Product gallery, Variant picker, Reviews

Orders tracking, Saved addresses, Wishlist

API Client: authService, orderService, couponService

AlertService, formatters, validation rules

ASP.NET Core Web API Backend

REST Controllers (OrdersController, CouponController)

DTOs, Request/Response payloads, ViewModels

CouponValidation, BillCalculation, TokenService

Database Context & Migrations

Uploaded media (WebP images, PDF documents)

3. End-to-End Data & Execution Flows

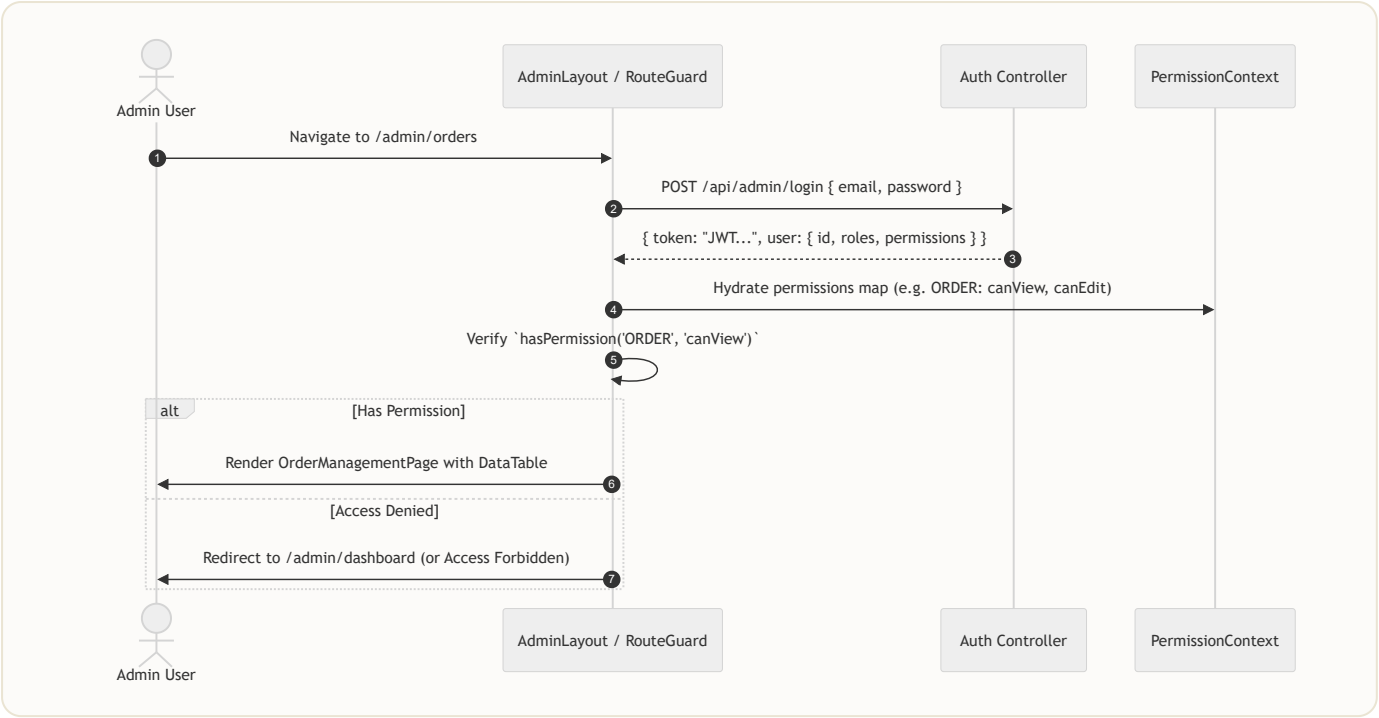
A. Order Placement & Bill Calculation Flow

```
sequenceDiagram
    actor Client as Client (Browser)
    participant Cart as Cart / Checkout State
    participant Orders as Orders API Controller
    participant Validator as Coupon & Stock Validator
    participant DB as SQL Database

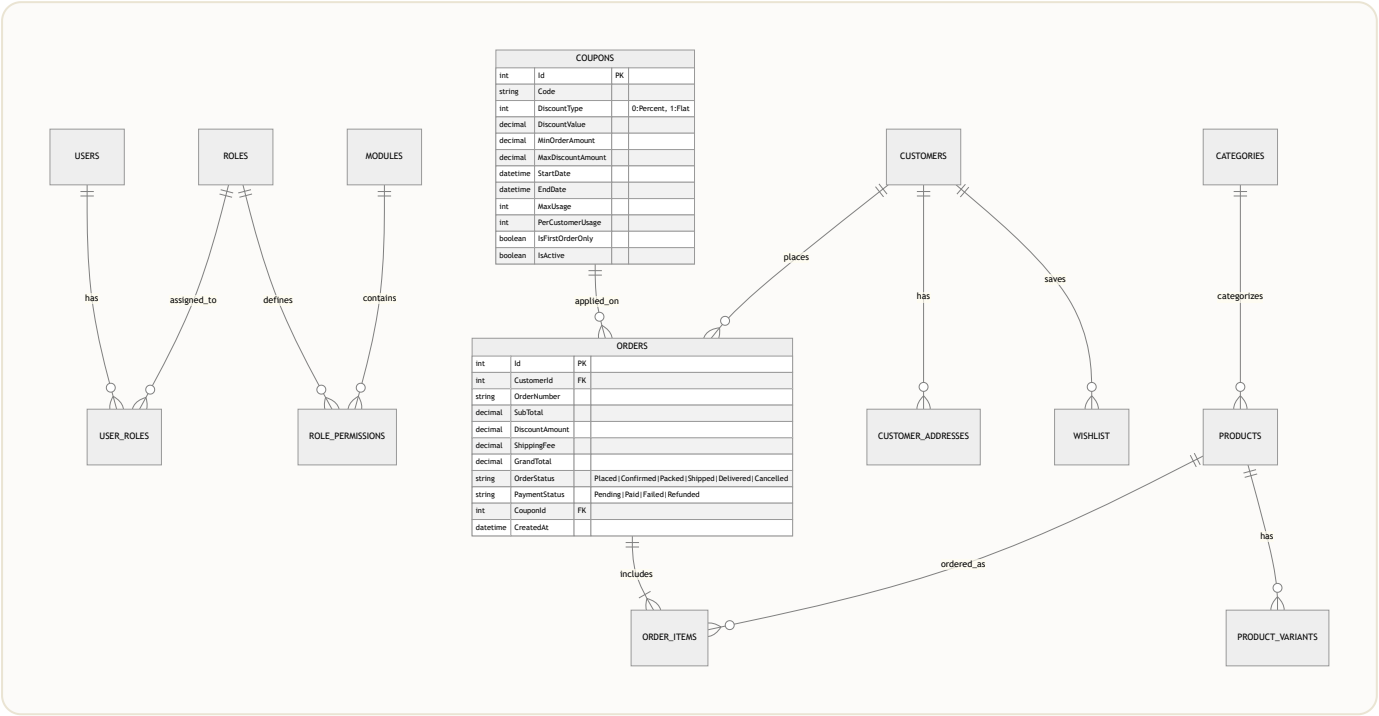
    Client->>Cart: 1 Add Item / Select Variant / Enter Coupon Code
    activate Cart
    Cart->>Orders: 2 POST /api/cart/calculate-bill [ Items, couponCode, addressId ]
    deactivate Cart
    activate Orders
    Orders->>Validator: 3 Validate Stock & Active Status
    Orders->>Validator: 4 Validate Coupon (MinOrder, MaxDiscount, Expiry, UserLimit)
    Validator-->>Orders: 5 Validated (DiscountAmount, ShippingFee, GrandTotal)
    deactivate Validator
    Orders-->>Cart: 6 200 OK (Calculated Bill Breakdown)
    deactivate Orders
    activate Cart
    Client->>Cart: 7 Click "Place Order" (COD / Online Payment)
    deactivate Client
    activate Cart
    Cart->>Orders: 8 POST /api/orders/checkout [ orderPayload ]
    deactivate Cart
    activate Orders
    Orders->>DB: 9 Begin Transaction (Insert Order, Insert OrderItems, Decrement Stock)
    DB-->>Orders: 10 Commit Transaction (OrderId: 1042)
    deactivate DB
    Orders-->>Client: 11 201 Created [ success: true, orderId: 1042 ]
    deactivate Orders
    deactivate Cart
```

The diagram illustrates the end-to-end flow for placing an order and calculating the bill. It involves the Client (Browser), Cart / Checkout State, Orders API Controller, Coupon & Stock Validator, and SQL Database. The process starts with the client adding items or selecting variants and entering a coupon code. The cart state then sends a POST request to the Orders API Controller to calculate the bill. The controller validates the stock and active status, and the coupon. The validator returns the validated details (DiscountAmount, ShippingFee, GrandTotal) to the controller. The controller then sends a 200 OK response (Calculated Bill Breakdown) back to the cart. The client then clicks "Place Order" (COD / Online Payment), which triggers the cart to send a POST request to the Orders API Controller for checkout. The controller begins a transaction (Insert Order, Insert OrderItems, Decrement Stock) and commits it (Commit Transaction (OrderId: 1042)) to the SQL Database. Finally, the controller sends a 201 Created response (success: true, orderId: 1042) back to the client.

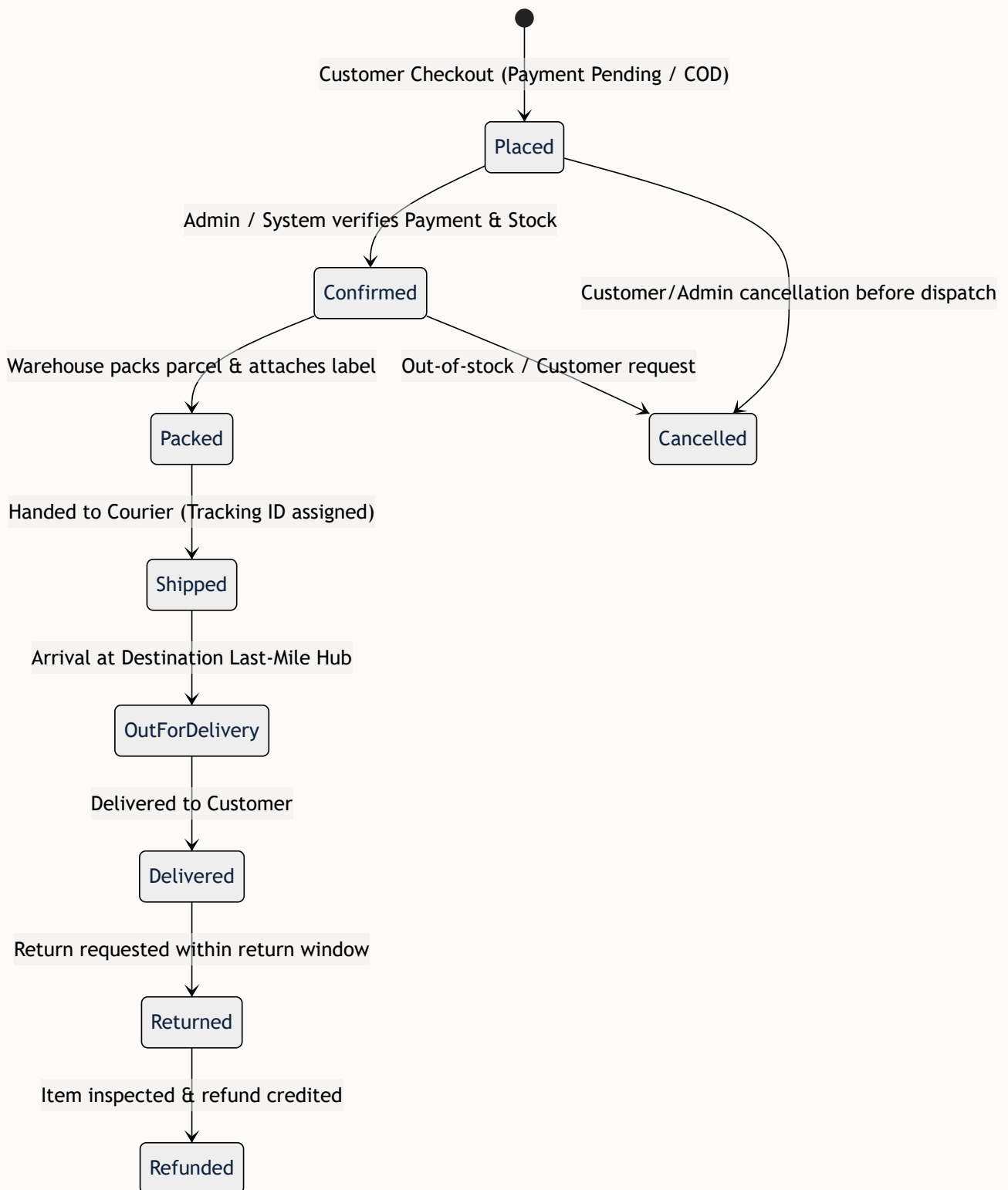
B. Authentication & Role-Based Access Control (RBAC) Flow



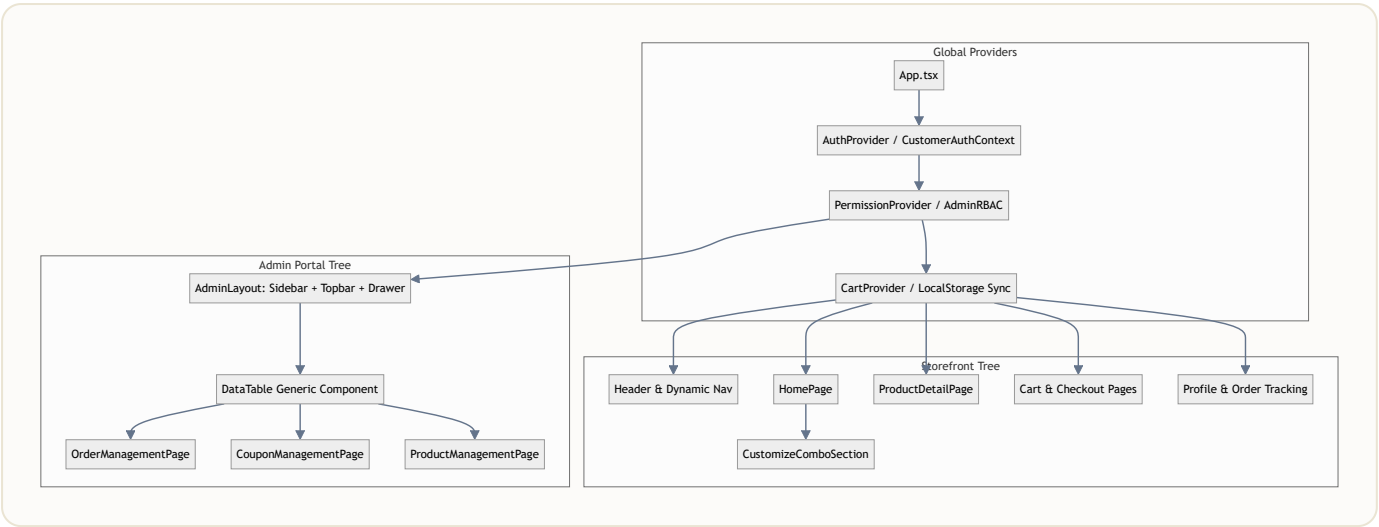
4. Core Database Schema & Relationships



5. State Machine: Order Lifecycle



6. Frontend Component & State Architecture



7. Automated Testing Strategy (Playwright E2E Matrix)

The test harness in `hiya2.client/e2e-tests/` verifies 38 critical end-to-end paths across storefront and admin:

Suite / Spec File	Coverage Areas	Verification Assertions
----	----	----
<code>all-9-excel-bugs.spec.ts</code>	Bug fixes OC-01 through OC-09	Upload controls, order status filters, cart login CTA, accessible sort
<code>matrix-38-cases.spec.ts</code>	Complete QA Matrix (38 test cases)	Pricing, combo builder, coupon boundary conditions, inventory sync
<code>qa-executive-summary.spec.ts</code>	Executive regression test	High-level smoke test across checkout, auth, and order tracking

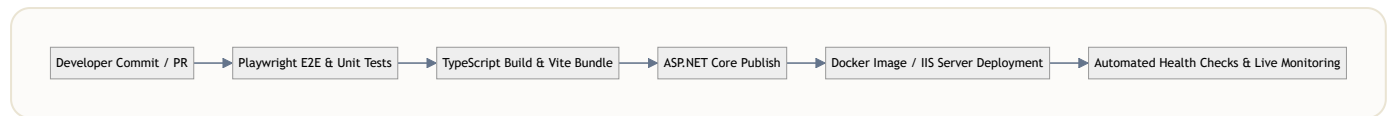
Running the E2E Test Suite:

```
# Run all automated test suites headless
npx playwright test

# Run with interactive Playwright UI Runner
npx playwright test --ui

# View generated test trace report
npx playwright show-report
```

8. CI/CD & Deployment Pipeline



9. Next Sprint Engineering Backlog

1. Webhook Handler for Payment Gateway:

- Webhook endpoint (`/api/payments/razorpay-webhook`) to verify signature and mark orders **Paid** asynchronously.

2. Third-party Logistics (3PL) API:

- Integration with Shiprocket/Delhivery API for automatic manifest creation and real-time webhook status polling.

3. PWA & Offline Service Worker:

- Add manifest and caching strategy for offline product browsing and push notifications.